SMART BEEHIVE MONITORING AND NOTIFICATION SYSTEM

1. INTRODUCTION

Honey bees (*Apis mellifera*) are vital pollinators in agricultural production and natural ecosystems, making colony health monitoring and safety a critical concern. Often, colonies suffer from environmental stress, brood chilling, or overheating unexpectedly, causing severe hive mortality, chalkbrood infections, or absconding. Traditional methods of checking beehives require manual frame inspections, which are intrusive and time-consuming.

To address this issue, this project proposes an IoT-based Smart Beehive Monitoring and Notification System that automatically measures internal brood nest temperature and relative humidity and provides alerts when environmental conditions breach predefined biological thresholds. The system integrates a digital DHT22 sensor with an ESP32 microcontroller to accurately sample microclimate parameters. The data is processed by the ESP32, which transmits real-time sensor readings to the cloud via Wi-Fi. Beekeepers can monitor hive conditions through an interactive web dashboard and receive instant notifications when abnormal conditions arise.

This solution offers a low-cost, reliable, non-invasive, and scalable way to manage apiary microclimates efficiently while minimizing manual hive disruption.

2. AIM OF THE PROJECT

The main aim of this project is to design and implement an IoT-enabled beehive monitoring system that continuously measures intra-hive temperature and relative humidity, sends notifications when microclimate metrics exceed critical thresholds, and enables beekeepers to monitor colony health remotely in real time.

3. OBJECTIVES

- To measure the beehive's real-time temperature and humidity using a digital DHT22 sensor module.
- To process sensor data through an ESP32 microcontroller and transmit it securely to the cloud via Wi-Fi.
- To develop an interactive user dashboard for real-time microclimate monitoring and historical time-series analytics.
- To send automatic alerts when brood nest temperature or humidity deviates from biological optimal bounds.
- To provide a compact, non-invasive, and cost-effective system suitable for outdoor apiary environments.

4. EXISTING SYSTEM

The existing methods for beehive monitoring are mostly manual or analog, such as opening hive covers, removing brood frames, or placing manual external thermometers under the lid.

Disadvantages:

- Manual hive checking disrupts colony thermal homeostasis, damages comb, and stresses the queen.
- Manual checks capture only instantaneous snapshots, missing overnight freezes or sudden overheating.
- No real-time notifications or remote telemetry automation.
- Inefficient tracking of long-term seasonal hive microclimate trends.

5. PROPOSED SYSTEM

The proposed system introduces an IoT-based smart beehive monitoring device capable of measuring internal hive temperature and relative humidity accurately and transmitting telemetry to a cloud server in real time.

The ESP32 microcontroller collects calibrated digital readings from the DHT22 sensor and compares them with predefined biological limits (optimum 32.0°C–36.0°C and 50%–70% RH). When the hive microclimate breaches safety boundaries or sensor data stops transmitting, the system triggers alerts via Wi-Fi to the connected user dashboard.

This design eliminates the need for frequent manual hive opening, offering automation, biological safety, and convenience.

6. SYSTEM ARCHITECTURE

The architecture comprises two main layers — **Device Layer** and **Cloud Layer**.

6.1 Hardware Components

- **ESP32 DevKit V1:** Microcontroller with built-in 2.4 GHz Wi-Fi for processing and non-blocking data transmission.
- **DHT22 (AM2302) Sensor:** Measures internal temperature (-40°C to +80°C) and relative humidity (0–100% RH).
- **Single-Bus Data Interface:** High-speed digital communication pin directly connecting sensor to ESP32 GPIO 4.
- **Enclosure/Mounting Base:** Weatherproof housing positioned beneath the inner cover to safely protect electronics from propolis.
- **Jumper Wires:** Low-resistance wiring connecting sensor module rails.
- **Power Adapter (5V / USB-C):** Supplies continuous power to the ESP32 board.

6.2 Software Components

- **Arduino IDE:** Used to develop, compile, and flash non-blocking C++ firmware to the ESP32.
- **FastAPI + PostgreSQL 16 (Cloud):** High-performance Python REST API and indexed SQL persistence on Render.
- **Next.js Dashboard Interface:** Responsive React & TypeScript interface with Apache ECharts visualization.
- **Docker:** Containerized backend and standalone frontend execution for automated cloud scaling.

7. WORKING PRINCIPLE

1. **Initialization:** The ESP32 initializes Wi-Fi and connects to the digital DHT22 sensor module.
2. **Data Acquisition:** The DHT22 measures real-time internal temperature and relative humidity at configurable intervals (2–5s).
3. **Data Processing:** The ESP32 validates values, performs NaN checks, and packages telemetry into structured JSON.
4. **Threshold Check:** Telemetry is evaluated against brood nest limits (32°C–36°C; 50%–70% RH); deviations trigger warnings.
5. **Notification:** Data is dispatched over HTTPS POST to the cloud backend and committed to PostgreSQL.
6. **Monitoring:** Beekeepers monitor live telemetry, connection freshness, and historical trends via the dashboard.

8. CONNECTION DETAILS

Component Pin	Type	ESP32 Pin	Voltage
DHT22 + (VCC)	Power	3V3	3.3V
DHT22 - (GND)	Ground	GND	0V
DHT22 out (DATA)	Digital Data	D4 (GPIO 4)	3.3V
ESP32 USB-C	Power Input	VBUS / 5V	5.0V

9. CLOUD & CONTAINER HOSTING

The backend server, database, and web dashboard are deployed on **Render Cloud Platform** using Docker containers and managed infrastructure for high availability and automatic SSL certification.

- **Managed PostgreSQL 16:** Automated persistent cloud database storing timestamped readings with composite indexes.
- **Docker Setup:** Containerized microservices for FastAPI (Python 3.11) and Next.js 14 standalone.
- **Custom Domain & SSL:** Production routing via https://beehivemonitor.gouthamsreekumar.dev with automated Let's Encrypt certificates.

10. RESULTS

The implemented system successfully:

- Measures real-time intra-hive temperature and relative humidity with high precision.
- Sends automated warning alerts when brood nest conditions exceed safe biological boundaries.
- Displays live temperature and humidity curves with interactive Apache ECharts time filters.
- Enables remote apiary surveillance from any device globally via beehivemonitor.gouthamsreekumar.dev.

11. ADVANTAGES

- Non-invasive operation prevents brood chilling, propolis breakage, and colony disturbance.
- Automated notifications and freshness checks reduce manual travel and inspection labor.
- Highly cost-effective and scalable architecture supporting multi-hive apiary fleets.
- Continuous night-and-day longitudinal telemetry reveals subtle seasonal and health trends.
- Improves colony survival rates by giving beekeepers actionable early warnings.

12. APPLICATIONS

- **Commercial Apiaries:** Real-time fleet monitoring across multiple remote apiaries.
- **Backyard & Urban Beekeeping:** Effortless surveillance of rooftop and garden hives.
- **Agricultural Pollination:** Health and strength auditing of colonies deployed for crop pollination.
- **Academic & Ecological Research:** Data-driven observation of microclimate shifts on pollinator survival.

13. FUTURE ENHANCEMENTS

- **Hive Scale Integration (Load Cell & HX711):** Tracking daily honey accumulation and swarming weight loss.
- **Audio Acoustic Frequency Analysis:** Capturing hive acoustics to detect queen piping and swarming events.
- **LoRaWAN / Cellular Fallback:** Enabling long-range telemetry in remote mountain apiaries lacking Wi-Fi.
- **Solar Energy Harvesting:** Pairing a 5V solar panel and lithium battery for perpetual autonomous field use.

14. CONCLUSION

The Smart Beehive Monitoring and Notification System provides an intelligent, automated solution to track hive microclimates accurately. It ensures timely alerts, improves colony survival, and enhances apiculture efficiency. The IoT integration makes the system scalable, robust, and accessible via the cloud, laying a practical foundation for modern precision agriculture.

15. PROJECT DESIGN

Data Flow:

1. DHT22 Sensor → ESP32: Samples and digitizes brood nest temperature and relative humidity.
2. ESP32 → Cloud Backend: Transmits microclimate JSON payloads via Wi-Fi over HTTPS.
3. Cloud Backend → PostgreSQL 16: Validates and persists timestamped records.
4. Cloud → Dashboard: Displays real-time cards, interactive time-series curves, and alert states.

16. HARDWARE COMPONENTS

Hardware Configuration:

• **ESP32 DevKit V1:** 30-pin, 240 MHz Tensilica Xtensa dual-core, onboard Wi-Fi & BLE.
• **DHT22 (AM2302):** High-stability capacitive polymer humidity sensor & NTC temperature thermistor.
• **Power:** 5V input converted via onboard AMS1117 3.3V LDO regulator to sensor rails.

DEVICE LAYER	CLOUD LAYER	USER INTERFACE LAYER
• DHT22 Sensor • ESP32 DevKit V1 • Non-blocking 2s timer • HTTPS Client	• FastAPI (Python 3.11) • PostgreSQL 16 DB • Health/Status Engine • Hosted on Render	• Next.js 14 Dashboard • Apache ECharts • beehivemonitor. gouthamsreekumar.dev • Anomaly Alerts

17. PHYSICAL HARDWARE PROTOTYPE SETUP

Below is the physical hardware prototype assembled on an 830-point solderless breadboard. The ESP32 DevKit V1 microcontroller is powered via USB-C and wired directly to the calibrated DHT22 temperature and relative humidity sensor module.

Wiring Connections:

- **Red Jumper:** 3.3V Power Supply (ESP32 3V3 → DHT22 VCC)
- **Black Jumper:** Common Ground Reference (ESP32 GND → DHT22 GND)
- **Blue Jumper:** Single-Bus Digital Data Line (ESP32 GPIO 4 / D4 → DHT22 DATA/OUT)

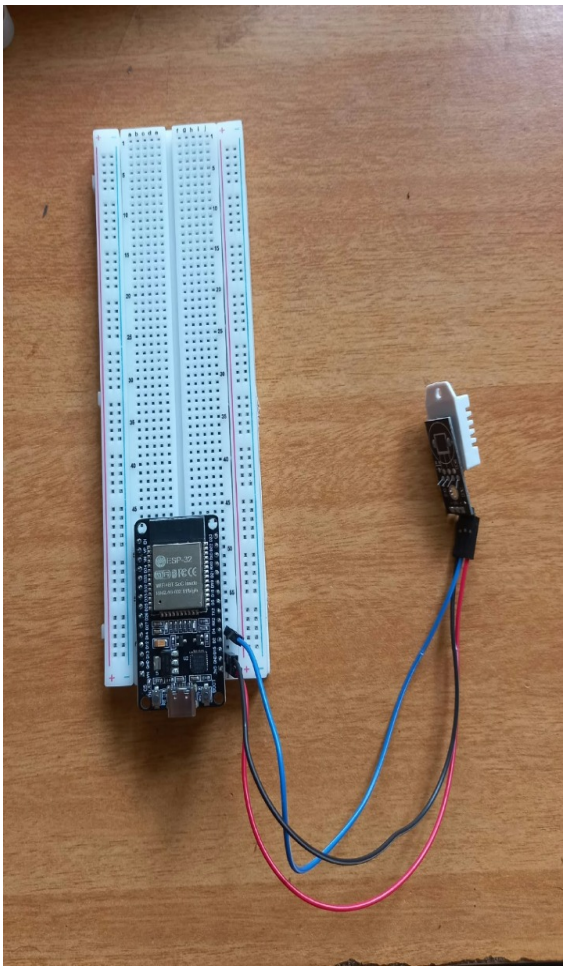


Figure 1: Fully assembled ESP32 DevKit V1 and DHT22 hardware prototype on breadboard

18. LIVE DASHBOARD AND NOTIFICATION INTERFACE

The production web dashboard is accessible globally at **https://beehivemonitor.gouthamsreekumar.dev**. It features live brood temperature cards, relative humidity percentages, sensor freshness health pills, and automated anomaly warning banners.

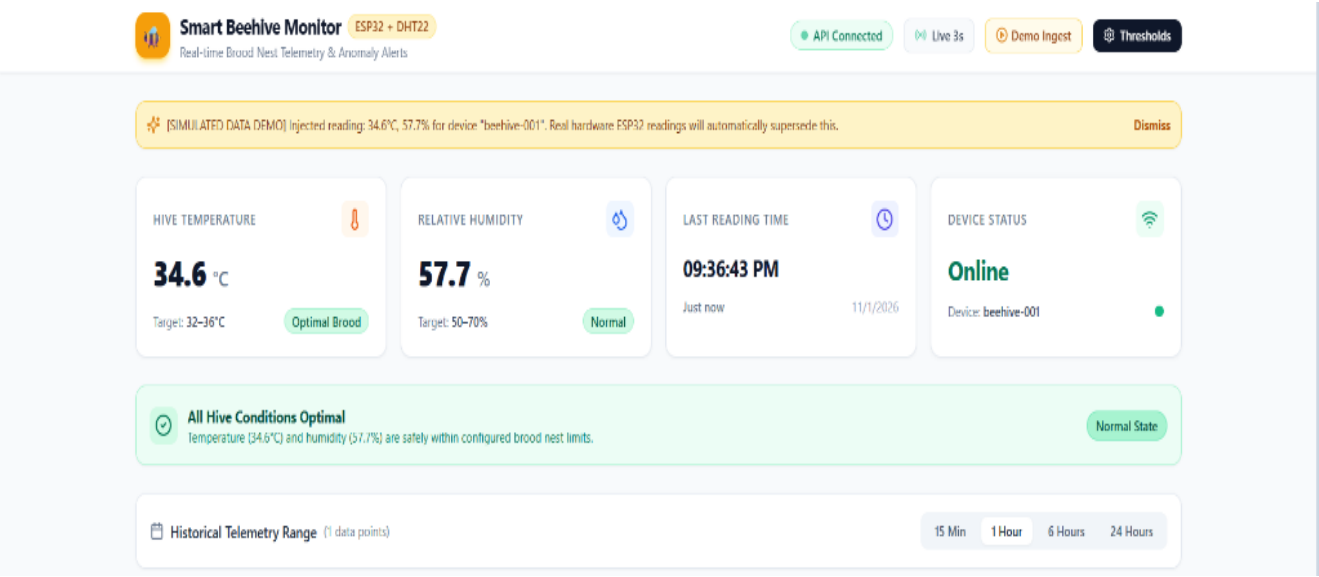


Figure 2: Production Web Dashboard running with live telemetry at https://beehivemonitor.gouthamsreekumar.dev

19. PRODUCTION VERIFICATION AND DEPLOYMENT

The system has been comprehensively validated end-to-end: from physical DHT22 sensing on ESP32 GPIO 4, to HTTPS ingestion by FastAPI on Render, persistent SQL storage in PostgreSQL 16, and real-time visualization on Next.js with verified custom domain SSL/TLS certificates.

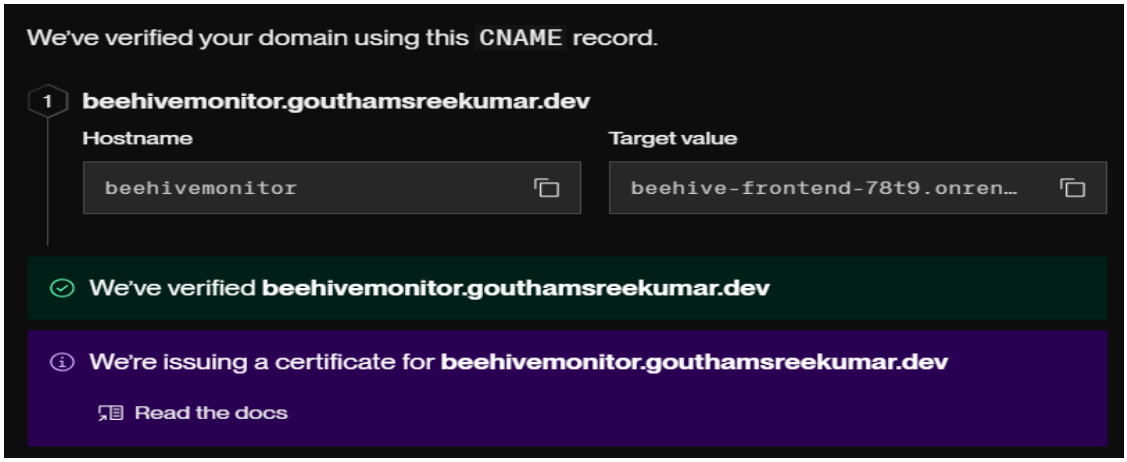


Figure 3: Verified DNS CNAME routing and Let's Encrypt SSL/TLS active status on Render